

PRAKTIK KECERDASAN BUATAN
MODUL PRAKTIKUM: INFORMED (HEURISTIC) SEARCH STRATEGIES



Disusun Oleh:

Zalfa Deizrantio Hamboro

24090620067

Dosen Pengampu:

Alfiyah Qothrunnada Zulfa S.Pd., M.Eng.

DEPARTEMEN TEKNIK ELEKTRO DAN ELEKTRONIKA

FAKULTAS VOKASI

UNIVERSITAS NEGERI YOGYAKARTA

2026

I. Tujuan Praktikum

- Mahasiswa mampu mendefinisikan dan menggunakan fungsi heuristik $h(n)$ dalam program pencarian.
- Mahasiswa mampu mengimplementasikan Greedy Best-First Search dan A* Search..
- Mahasiswa mampu menjelaskan konsep admissibility dan consistency pada heuristik, serta membandingkan efisiensi informed search terhadap uninformed search yang telah dipelajari pada Minggu 2.

II. Dasar Teori Singkat

Informed (heuristic) search adalah metode pencarian yang menggunakan fungsi heuristik $h(n)$, yaitu perkiraan biaya dari suatu node menuju tujuan (goal). Penggunaan heuristik membantu pencarian menjadi lebih terarah dan efisien karena algoritma dapat memilih node yang dianggap lebih dekat dengan tujuan. Dua algoritma utama dalam metode ini adalah Greedy Best-First Search dan A* Search. Greedy Best-First Search menggunakan fungsi $f(n) = h(n)$, sehingga hanya mempertimbangkan perkiraan jarak menuju tujuan. Metode ini biasanya lebih cepat, tetapi tidak menjamin mendapatkan jalur dengan biaya paling rendah karena tidak memperhitungkan biaya perjalanan yang sudah ditempuh $g(n)$.

Sementara itu, A* Search menggunakan fungsi $f(n) = g(n) + h(n)$, sehingga mempertimbangkan biaya yang sudah ditempuh sekaligus perkiraan biaya yang masih diperlukan untuk mencapai tujuan. A* dapat menghasilkan solusi yang optimal apabila heuristik yang digunakan bersifat admissible, yaitu tidak pernah melebihi-lebihkan biaya sebenarnya dari suatu node menuju goal ($h(n) \leq h^*(n)$). Pada percobaan ini digunakan heuristik straight-line distance, yang bersifat admissible karena jarak garis lurus antara dua kota tidak mungkin lebih jauh daripada jarak perjalanan sebenarnya melalui jalan.

III. Alat dan Bahan

- Python 3.18 ke atas.
- Modul heapq.
- Variable graph dari percobaan 1 Minggu 2.

IV. Langkah Percobaan

Percobaan 1 – Fungsi Heuristik

```
# Fungsi heuristik: estimasi jarak garis lurus (straight-line distance)
# dari setiap kota menuju Bucharest (AIMA Fig. 3.16)
h_sld = {
    'Arad': 366, 'Bucharest': 0, 'Craiova': 160, 'Drobeta': 242,
    'Eforie': 161, 'Fagaras': 176, 'Giurgiu': 77, 'Hirsova': 151,
    'Iasi': 226, 'Lugoj': 244, 'Mehadia': 241, 'Neamt': 234,
    'Oradea': 380, 'Pitesti': 100, 'RimnicuVilcea': 193, 'Sibiu': 253,
    'Timisoara': 329, 'Urziceni': 80, 'Vaslui': 199, 'Zerind': 374,
}
# Catatan: h_sld(Bucharest) = 0, karena goal selalu memiliki estimasi 0.
# Heuristik ini admissible karena jarak garis lurus TIDAK PERNAH melebihi
# jarak tempuh sesungguhnya di jalan (yang berkelok-kelok).
```

Percobaan 2 – Greedy Best – First Search

```
def greedy_best_first(graph, start, goal, h):
    frontier = [(h[start], start)] # diurutkan HANYA oleh h(n), evaluation
                                   # function f(n) = h(n)
    parent = {}
    reached = {start}
    expanded = 0
    while frontier:
        _, node = heapq.heappop(frontier)
        expanded += 1
        if node == goal:
            return reconstruct(parent, goal), expanded
        for neighbor in graph.get(node, {}):
            if neighbor not in reached:
                reached.add(neighbor)
                parent[neighbor] = node
                heapq.heappush(frontier, (h[neighbor], neighbor))
    return None, expanded

path, expanded = greedy_best_first(graph, 'Arad', 'Bucharest', h_sld)
print("Lintasan Greedy :", " -> ".join(path))
print("Node dieksplorasi:", expanded)
```

Percobaan 3 – A* Search

```
def a_star(graph, start, goal, h):
    frontier = [(h[start], 0, start)] # f(n) = g(n) + h(n)
    parent = {}
    best_cost = {start: 0}
    expanded = 0
    while frontier:
        f, g, node = heapq.heappop(frontier)
        if g > best_cost.get(node, float('inf')):
            continue
        expanded += 1
        if node == goal:
            return reconstruct(parent, goal), g, expanded
        for neighbor, step_cost in graph.get(node, {}).items():
            new_g = g + step_cost
            if new_g < best_cost.get(neighbor, float('inf')):
                best_cost[neighbor] = new_g
                parent[neighbor] = node
                heapq.heappush(frontier, (new_g + h[neighbor], new_g, neighbor))
    return None, None, expanded

path, cost, expanded = a_star(graph, 'Arad', 'Bucharest', h_sld)

print("Lintasan A* :", " -> ".join(path))
print("Total biaya :", cost)
print("Node dieksplorasi:", expanded)
```

V. Tabel Perbandingan Lima Algoritma

Lengkapi tabel berikut menggunakan hasil dari Minggu 2 (BFS, DFS, UCS) dan Minggu 3 (Greedy, A*) untuk kasus Arad => Bucharest.

Algoritma	Lintasan	Total Biaya	Node Dieksplorasi	Optimal?
BFS	Arad → Sibiu → Fagaras → Bucharest	450	9	Tidak
DFS	Arad → Timisoara → Lugoj → Mehadia → Drobeta → Craiova → Pitesti → Bucharest	733	8	Tidak
UCS	Arad → Sibiu → RimnicuVilcea → Pitesti → Bucharest	418	13	Ya
Greedy Best-First	Arad → Sibiu → Fagaras → Bucharest	450	4	Tidak
A*	Arad → Sibiu → RimnicuVilcea → Pitesti → Bucharest	418	6	Ya

Berikut ini merupakan hasil dari program yang berjalan:

```

1. BFS
  Lintasan      : Arad -> Sibiu -> Fagaras -> Bucharest
  Total biaya   : 450
  Node dieksplorasi : 9
  Optimal?      : Tidak

2. DFS
  Lintasan      : Arad -> Timisoara -> Lugoj -> Mehadia -> Drobeta -> Craiova -> Pitesti -> Bucharest
  Total biaya   : 733
  Node dieksplorasi : 8
  Optimal?      : Tidak

3. UCS
  Lintasan      : Arad -> Sibiu -> RimnicuVilcea -> Pitesti -> Bucharest
  Total biaya   : 418
  Node dieksplorasi : 13
  Optimal?      : Ya

4. Greedy Best-First
  Lintasan      : Arad -> Sibiu -> Fagaras -> Bucharest
  Total biaya   : 450
  Node dieksplorasi : 4
  Optimal?      : Tidak

5. A*
  Lintasan      : Arad -> Sibiu -> RimnicuVilcea -> Pitesti -> Bucharest
  Total biaya   : 418
  Node dieksplorasi : 6
  Optimal?      : Ya

=====
BIAYA OPTIMAL = 418

```

Keterangan: Biaya optimal pada kasus Arad → Bucharest adalah 418. Algoritma dikatakan optimal apabila menghasilkan lintasan dengan total biaya 418. Dari hasil percobaan, UCS dan A* menghasilkan solusi optimal, sedangkan BFS, DFS, dan Greedy Best-First menghasilkan lintasan dengan biaya lebih tinggi. Greedy Best-First memiliki jumlah node yang dieksplorasi paling sedikit, yaitu 4 node, tetapi tidak optimal. A* mampu memperoleh biaya optimal dengan hanya mengeksplorasi 6 node, lebih sedikit dibandingkan UCS yang mengeksplorasi 13 node. Pada percobaan nilai estimasi total biaya dari BFS dan DFS diperoleh berdasarkan hasil dari lintasan kemudian di estimasi setelah lintasan berhasil diperoleh.

VI. Eksperimen Mandiri

1. Uji Greedy Best-First Search dan A* untuk pasangan kota lain, misalnya Iasi → Fagaras dan Neamt → Pitesti. Catat hasilnya pada tabel serupa B.5.

Iasi → Faragas

Algoritma	Lintasan	Total Biaya	Node Dieksplorasi	Optimal?
Greedy Best-First	Iasi -> Vaslui -> Urziceni -> Bucharest -> Fagaras	530	10	Ya
A*	Iasi -> Vaslui -> Urziceni -> Bucharest -> Fagaras	530	10	Ya

Neamt → Pitesti

Algoritma	Lintasan	Total Biaya	Node Dieksplorasi	Optimal?
Greedy Best-First	Neamt -> Iasi -> Vaslui -> Urziceni -> Bucharest -> Pitesti	507	7	Ya
A*	Neamt -> Iasi -> Vaslui -> Urziceni -> Bucharest -> Pitesti	507	7	Ya

Hasil Greedy Best-First Search dan A* yang sama pada kedua skenario tersebut dapat terjadi karena pada kasus yang diuji, kedua algoritma akhirnya memilih lintasan yang sama menuju tujuan. Greedy Best-First Search hanya mempertimbangkan nilai heuristik $h(n)$, sedangkan A* mempertimbangkan biaya yang sudah ditempuh $g(n)$ ditambah heuristik $h(n)$. Pada Iasi → Fagaras, keduanya menghasilkan lintasan Iasi → Vaslui → Urziceni → Bucharest → Fagaras dengan biaya 530 dan mengeksplorasi 10 node.

Sementara pada Neamt → Pitesti, keduanya menghasilkan lintasan Neamt → Iasi → Vaslui → Urziceni → Bucharest → Pitesti dengan biaya 507 dan mengeksplorasi 7 node. Berdasarkan tabel hasil pengujian, kedua lintasan tersebut dinyatakan optimal, sehingga tidak terdapat perbedaan hasil antara Greedy Best-First Search dan A* pada kedua skenario tersebut. Hal ini menunjukkan bahwa meskipun cara kerja kedua algoritma berbeda, pada kondisi tertentu keduanya dapat memilih jalur yang sama dan menghasilkan biaya yang sama.

2. Buat heuristik baru yang sengaja TIDAK admissible, misalnya $h_tidak_admissible = \{kota: nilai * 2 \text{ for } kota, nilai \text{ in } h_sld.items()\}$, lalu jalankan A* dengan heuristik tersebut untuk kasus Arad → Bucharest. Apakah lintasan yang ditemukan masih optimal (biaya 418)? Jelaskan mengapa demikian.

```
=====
Kasus           : Arad -> Bucharest
Lintasan        : Arad -> Sibiu -> Fagaras -> Bucharest
Total biaya     : 450
Node dieksplorasi : 4
Optimal?        : Tidak
Biaya optimal UCS : 418
=====
```

Tidak optimal karena berdasarkan hasil nilai, A* dijalankan untuk Arad → Bucharest, maka heuristik tersebut sengaja dibuat tidak admissible karena nilai heuristik menjadi dua kali lipat dari estimasi SLD. Akibatnya, nilai $h(n)$ dapat melebihi-lebihkan biaya sebenarnya menuju tujuan.

3. Implementasikan Weighted A* Search dengan evaluation function $f(n) = g(n) + W \times h(n)$ untuk $W = 1.5$ dan $W = 2$. Bandingkan jumlah node yang dieksplorasi dan total biaya solusi dengan A* biasa ($W = 1$) pada kasus Arad → Bucharest.

```
Weighted A* (W = 2)
Lintasan          : Arad -> Sibiu -> Fagaras -> Bucharest
Total biaya       : 450
Node dieksplorasi : 4
=====
```

Pada hasil pengujian Weighted A* dengan $W = 2$, algoritma menghasilkan lintasan Arad → Sibiu → Fagaras → Bucharest dengan total biaya 450 dan hanya mengeksplorasi 4 node. Jika dibandingkan dengan A* biasa yang menghasilkan biaya optimal 418, Weighted A* $W = 2$ memang melakukan eksplorasi lebih sedikit, tetapi jalur yang

diperoleh tidak optimal karena biayanya lebih besar dari 418. Hal ini terjadi karena semakin besar nilai W , semakin besar pengaruh heuristik $h(n)$ dalam menentukan node yang dipilih. Akibatnya, pencarian menjadi lebih cepat dan fokus menuju tujuan, tetapi dapat mengabaikan jalur yang sebenarnya memiliki biaya lebih rendah. Jadi, Weighted A* dengan $W = 2$ lebih efisien dalam jumlah node yang dieksplorasi, tetapi mengorbankan optimalitas solusi.

4. (Tantangan — Studi Kasus 8-Puzzle) Implementasikan A* Search untuk masalah 8-puzzle menggunakan heuristik Manhattan Distance (total jarak horizontal + vertikal setiap ubin dari posisi saat ini ke posisi tujuannya). Representasikan state sebagai tuple 9 angka, dengan angka 0 mewakili petak kosong.

```
=====
A* SEARCH - 8 PUZZLE
=====

State Awal:
1 2 3
4 0 6
7 5 8

State Tujuan:
1 2 3
4 5 6
7 8 0

Heuristik Manhattan Distance: 2
Jumlah langkah: 2
Node dieksplorasi: 3

Lintasan solusi:
Langkah 0
1 2 3
4 0 6
7 5 8

Langkah 1
1 2 3
4 5 6
7 0 8

Langkah 2
1 2 3
4 5 6
7 8 0
```

VII. Pertanyaan Analisis

1. Mengapa lintasan yang ditemukan Greedy Best-First Search pada Percobaan 2 tidak optimal, padahal jumlah node yang dieksplorasi lebih sedikit dibandingkan A*? Kaitkan jawaban Anda dengan evaluation function masing-masing algoritma?

Jawab: Greedy Best-First Search pada Percobaan 2 dapat menghasilkan lintasan yang tidak optimal karena algoritma ini hanya menggunakan fungsi evaluasi $f(n) = h(n)$, yaitu perkiraan biaya dari node menuju tujuan. Greedy tidak mempertimbangkan biaya yang sudah ditempuh $g(n)$. Akibatnya, algoritma cenderung memilih node yang terlihat paling dekat dengan tujuan meskipun jalur tersebut sebenarnya memiliki biaya perjalanan yang lebih besar. Pada hasil percobaan, Greedy dapat mengeksplorasi lebih sedikit node dibandingkan A*, tetapi menghasilkan biaya solusi yang lebih tinggi. Sebaliknya, A* menggunakan $f(n) = g(n) + h(n)$ sehingga mempertimbangkan biaya yang sudah ditempuh dan perkiraan biaya menuju tujuan. Oleh karena itu, A* dapat menemukan solusi yang lebih optimal meskipun jumlah node yang dieksplorasi lebih banyak.

2. Bandingkan jumlah node yang dieksplorasi oleh UCS (Minggu 2) dan A* (Minggu 3) untuk mencapai solusi optimal yang sama (biaya 418). Apa yang menyebabkan perbedaan efisiensi tersebut?

Jawab: Berdasarkan hasil percobaan, UCS mengeksplorasi 13 node, sedangkan A* hanya mengeksplorasi 6 node untuk mendapatkan lintasan optimal dengan biaya yang sama, yaitu 418. Perbedaan tersebut terjadi karena UCS hanya menggunakan biaya yang sudah ditempuh $g(n)$ dalam menentukan node berikutnya. Akibatnya, UCS perlu mengeksplorasi lebih banyak kemungkinan jalur sebelum menemukan biaya minimum. Sementara itu, A* menggunakan $g(n) + h(n)$, sehingga heuristik dapat memberikan perkiraan arah menuju tujuan dan membuat pencarian lebih terarah. Dengan heuristik yang admissible, A* tetap dapat memperoleh solusi optimal seperti UCS, tetapi dengan eksplorasi node yang lebih sedikit.

3. Jelaskan hubungan antara admissibility heuristik dengan jaminan cost-optimal pada A* Search, kaitkan jawaban Anda dengan hasil Eksperimen Mandiri nomor 2.

Jawab: Admissibility berhubungan langsung dengan jaminan optimalitas A*. Heuristik dikatakan admissible apabila nilai $h(n)$ tidak pernah melebihi biaya sebenarnya dari node tersebut menuju tujuan. Dengan kondisi tersebut, A* dapat menggunakan heuristik untuk

mengarahkan pencarian tanpa melewati jalur yang memiliki biaya paling rendah. Hal ini terlihat pada A* dengan heuristik SLD yang menghasilkan biaya optimal 418. Pada Eksperimen Mandiri nomor 2, heuristik sengaja dibuat tidak admissible dengan mengalikan nilai heuristik menjadi dua kali lipat. Heuristik tersebut dapat melebihi biaya sebenarnya, sehingga A* tidak lagi memiliki jaminan cost-optimal. Jika pada percobaan tertentu hasilnya masih 418, hal tersebut hanya menunjukkan bahwa solusi optimal masih ditemukan pada kasus tersebut, bukan berarti heuristik tidak admissible tetap menjamin optimalitas.

4. Pada aplikasi navigasi nyata seperti Google Maps atau Waze, mengapa algoritma sejenis A* lebih disukai dibandingkan uniform-cost search murni?

Jawab: Pada aplikasi navigasi nyata seperti Google Maps atau Waze, algoritma sejenis A* lebih disukai dibandingkan Uniform-Cost Search murni karena pencarian perlu dilakukan secara cepat dan efisien pada jaringan jalan yang sangat besar. UCS hanya mempertimbangkan biaya yang sudah ditempuh sehingga dapat mengeksplorasi banyak jalan yang sebenarnya tidak mengarah secara efektif ke tujuan. A* menggunakan heuristik untuk memperkirakan jarak menuju tujuan sehingga pencarian menjadi lebih terarah dan jumlah node yang perlu dieksplorasi dapat dikurangi. Dengan heuristik yang tepat, algoritma seperti A* dapat memperoleh rute yang baik atau optimal dengan waktu pencarian yang lebih efisien.

VIII. Kesimpulan

Berdasarkan percobaan yang telah dilakukan, dapat disimpulkan bahwa setiap metode pencarian memiliki cara kerja dan kelebihan yang berbeda dalam menemukan lintasan dari satu kota ke kota tujuan. BFS (Breadth-First Search) mencari berdasarkan tingkat kedalaman sehingga dapat menemukan lintasan dengan jumlah langkah yang sedikit, tetapi belum tentu menghasilkan biaya perjalanan yang paling rendah. DFS (Depth-First Search) mencari dengan menyusuri satu jalur terlebih dahulu sampai dalam, sehingga dapat menemukan solusi dengan cepat pada kondisi tertentu, tetapi lintasan yang dihasilkan tidak menjamin menjadi lintasan dengan biaya paling rendah. UCS (Uniform Cost Search) memilih jalur berdasarkan biaya perjalanan yang sudah ditempuh, sehingga dapat menemukan lintasan dengan biaya minimum, tetapi membutuhkan eksplorasi node yang lebih banyak. Greedy Best-First Search menggunakan nilai heuristik untuk memilih node yang dianggap paling dekat dengan tujuan.

Metode ini dapat mengeksplorasi node lebih sedikit dan bekerja lebih cepat, tetapi tidak menjamin mendapatkan lintasan yang optimal. Sementara itu, A* menggabungkan biaya yang sudah ditempuh dengan perkiraan biaya menuju tujuan menggunakan fungsi ($f(n)=g(n)+h(n)$). Dengan heuristik yang admissible, A* dapat memperoleh lintasan optimal dengan jumlah eksplorasi yang lebih sedikit dibandingkan UCS. Oleh karena itu, A* dapat dikatakan sebagai metode yang lebih seimbang karena tetap memperhatikan biaya perjalanan sekaligus menggunakan informasi perkiraan jarak menuju tujuan.